LAPORAN PRAKTIKUM

POSTTEST (5)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Ferly Ahmad Nabil (2509106024)

Kelas (A2 ‘25)

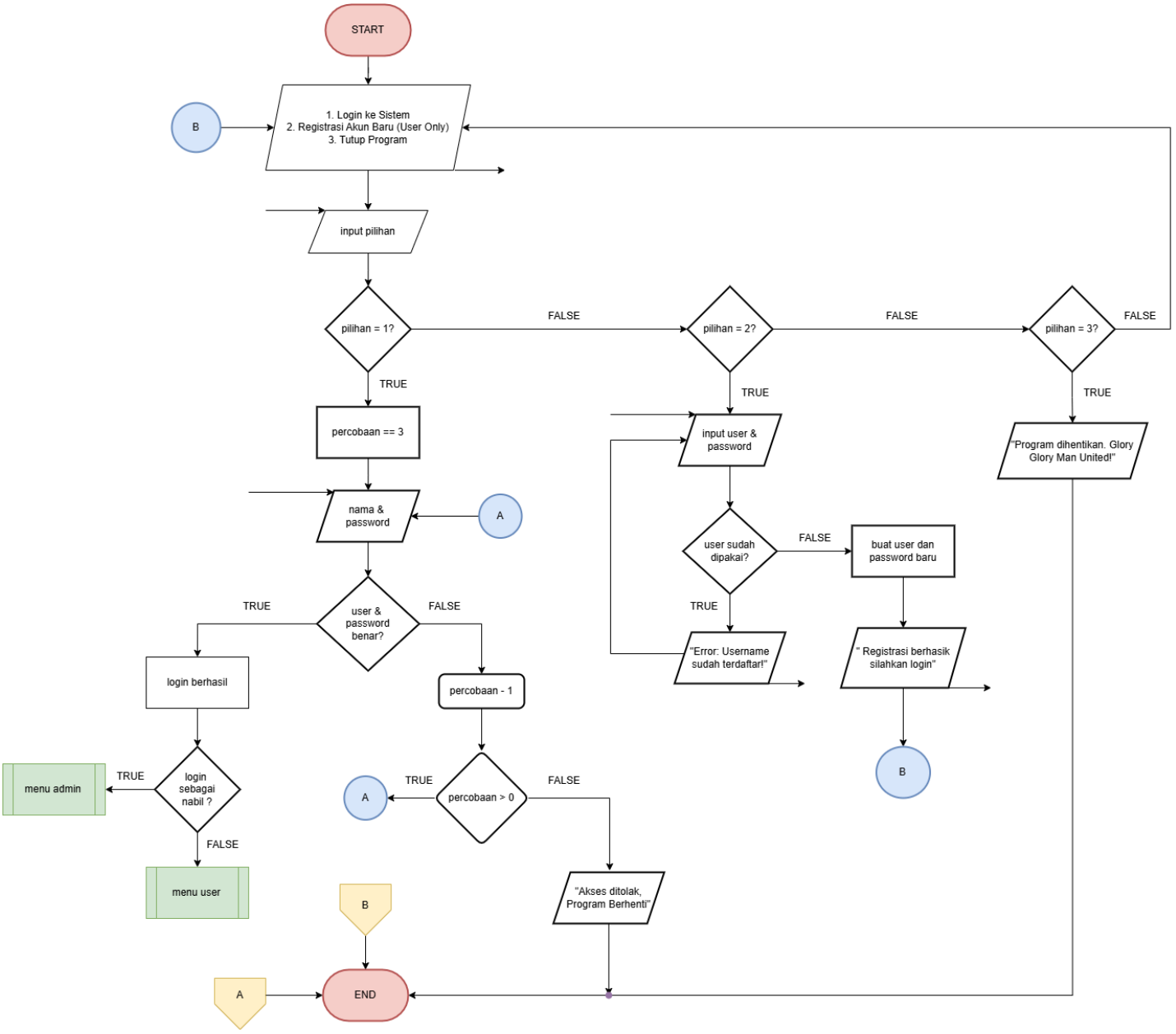
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

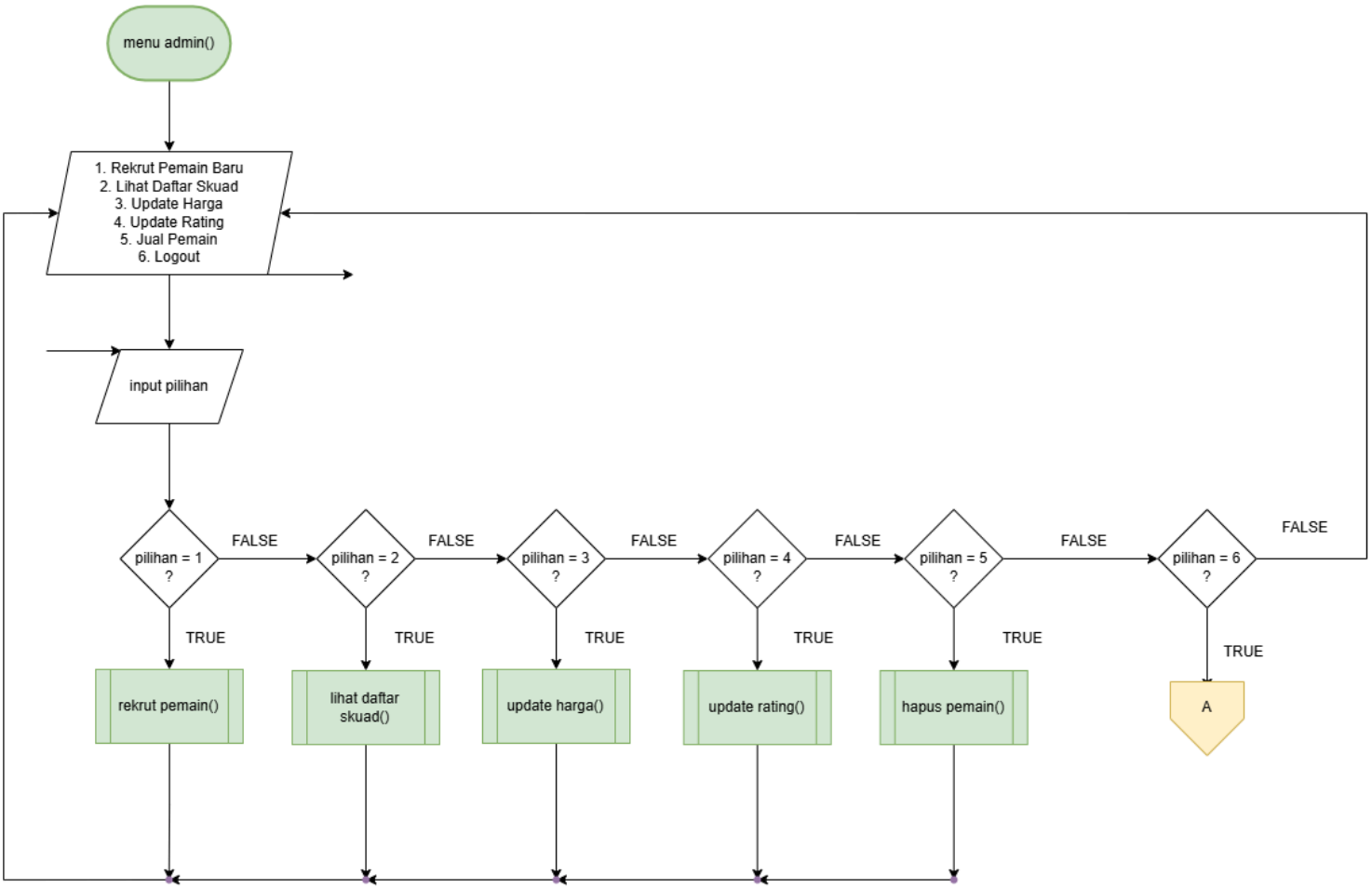
2026

1. Flowchart



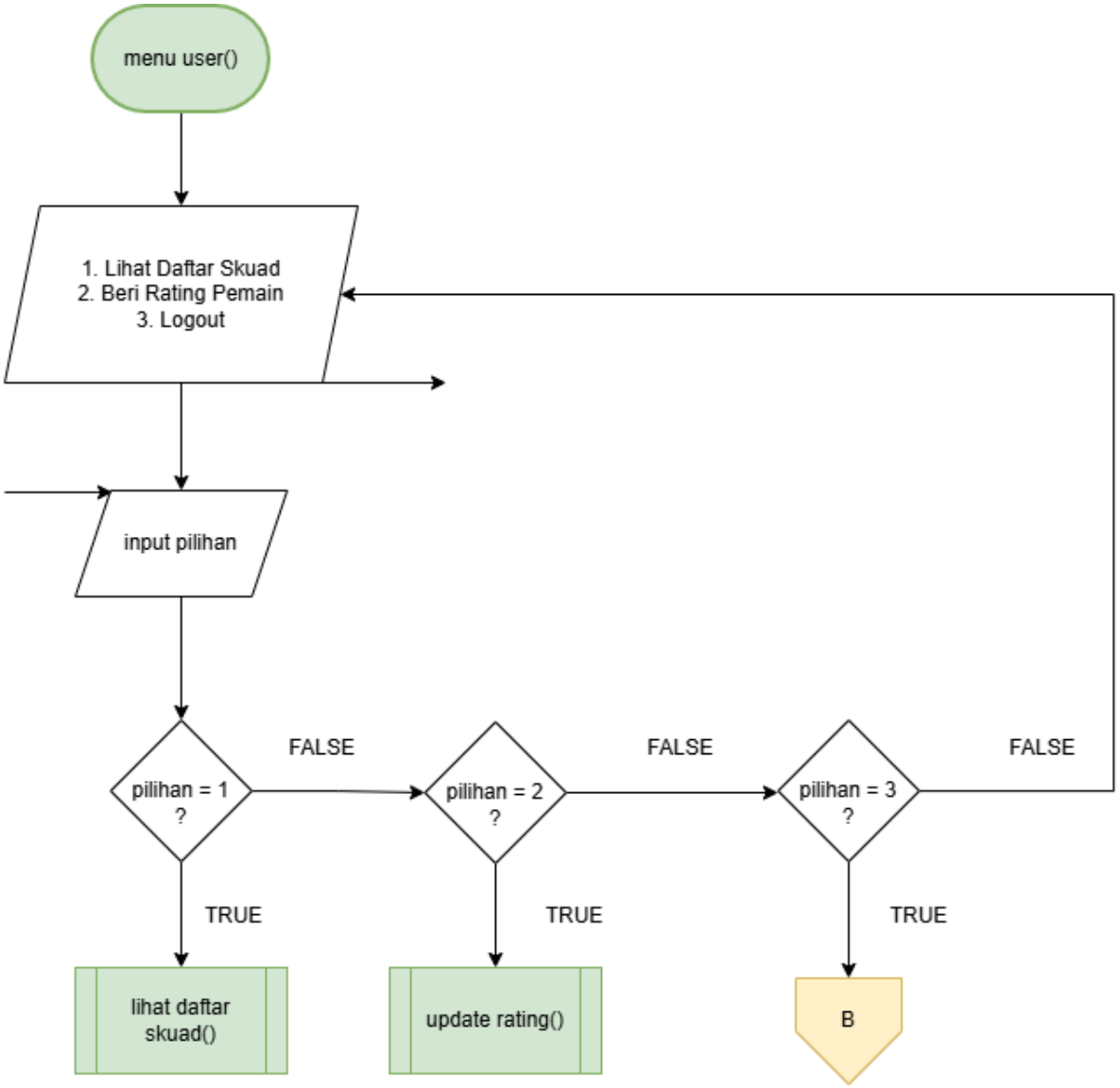
Gambar 1.1
<Flowchart Menu Utama Setelah Login>

Halaman ini mengatur akses masuk melalui tiga opsi utama, yaitu login, registrasi, dan keluar. Pengguna diberikan batas tiga kali percobaan login untuk memvalidasi akun sebagai admin (Nabil) atau user biasa; jika gagal melampaui batas tersebut, akses ditolak dan program berhenti. fitur login dan registrasi sudah memakai simbol predefined process (fungsi)



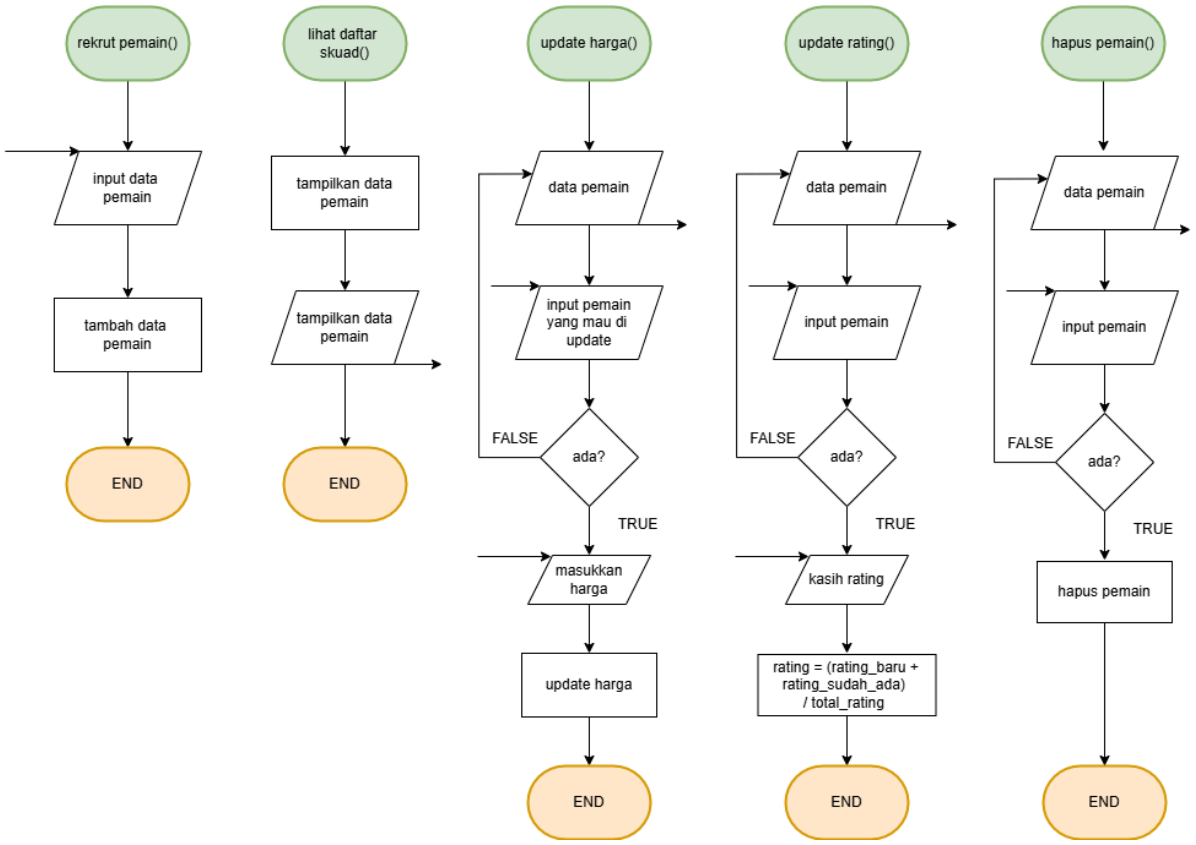
Gambar 1.2
<Flowchart Menu Adminn>

Bagian ini menguraikan hak akses penuh yang dimiliki oleh Admin dalam mengelola data pemain Manchester United. Admin dapat memilih enam opsi utama, mulai dari merekrut pemain baru dengan menginput data ke database, melihat daftar seluruh skuad, hingga melakukan pembaruan data spesifik seperti harga pasar dan rating performa pemain. Selain itu, Admin memiliki wewenang untuk menghapus data pemain dari sistem melalui menu "Jual Pemain", dan seluruh aktivitas ini berjalan dalam siklus yang akan terus kembali ke menu admin sampai pengguna memilih opsi logout untuk mengakhiri program.



Gambar 1.3
<Flowchart Menu User>

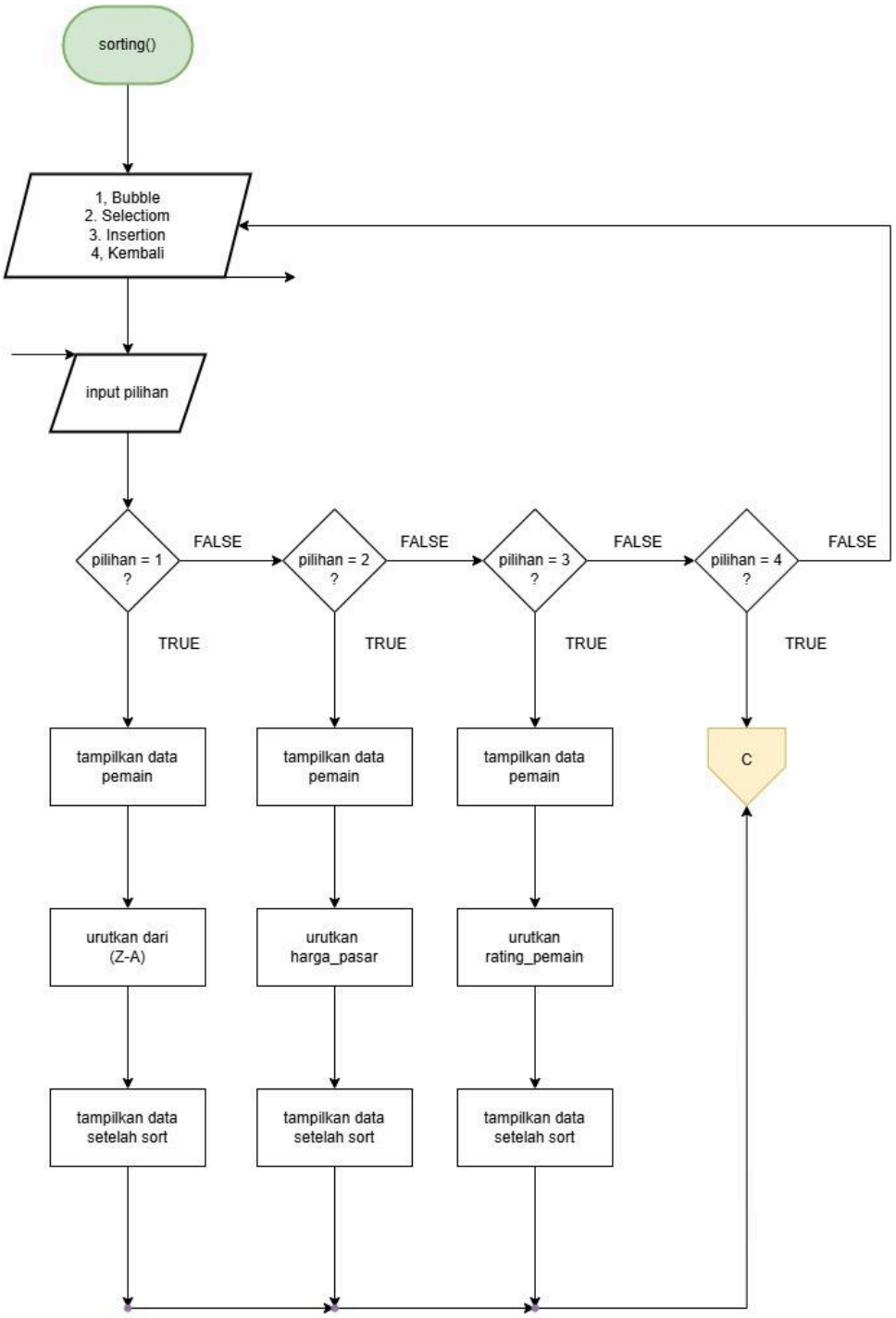
Bagian ini menyediakan akses terbatas bagi supporter yang hanya berfokus pada pemantauan dan pemberian apresiasi. Pengguna dapat melihat daftar skuad yang tersedia dan memberikan skor rating pada pemain tertentu, di mana sistem akan otomatis menghitung ulang rata-rata performa berdasarkan total penilai. Siklus menu ini akan terus berulang hingga pengguna memilih opsi *logout* untuk menyelesaikan sesi dan keluar dari aplikasi.



Gambar 1.4
<Flowchart Menu Fungsi>

Flowchart ini menggambarkan detail proses dari masing-masing fungsi yang dipanggil oleh menu admin, terdiri dari lima sub program:

- **rekrut pemain()** — Menerima input data pemain (nama, klub, negara, harga), kemudian menyimpannya ke dalam array data pemain, lalu END.
- **lihat daftar skuad()** — Menampilkan seluruh data pemain yang tersimpan dalam dua tahap output, lalu END.
- **update harga()** — Menampilkan data pemain, lalu meminta input nomor pemain yang ingin diupdate. Sistem mengecek apakah pemain tersebut ada — jika tidak (FALSE) langsung END; jika ada (TRUE), meminta input harga baru lalu menyimpannya dan END.
- **update rating()** — Menampilkan data pemain, lalu meminta input nomor pemain. Jika pemain tidak ada (FALSE) langsung END; jika ada (TRUE), meminta input rating baru dari pengguna, kemudian menghitung rating terbaru dengan rumus rating = (rating_baru + rating_sudah_ada) / total_rating, lalu END.
- **hapus pemain()** — Menampilkan data pemain, lalu meminta input nomor pemain yang akan dijual. Jika pemain tidak ada (FALSE) langsung END; jika ada (TRUE), menghapus data pemain tersebut dari array, lalu END.



Gambar 1.5
<Flowchart Menu Fungsi Sort>

Flowchart ini menggambarkan proses **sorting data pemain** yang dipilih melalui menu sorting(). Sistem menampilkan pilihan metode yaitu Bubble Sort, Selection Sort, Insertion Sort, dan Kembali, lalu pengguna memasukkan pilihan.

Jika memilih **1 (Bubble Sort)**, sistem menampilkan data pemain, mengurutkan berdasarkan **nama (Z–A)**, kemudian menampilkan hasil.

Jika memilih **2 (Selection Sort)**, sistem menampilkan data, mengurutkan berdasarkan **harga (termurah ke termahal)**, lalu menampilkan hasil.

Jika memilih **3 (Insertion Sort)**, sistem menampilkan data, mengurutkan berdasarkan **rating (tertinggi ke terendah)**, lalu menampilkan hasil.

Jika memilih **4 (Kembali)**, sistem langsung kembali ke menu sebelumnya tanpa melakukan sorting.

2. Deskripsi Singkat Program

Program ini merupakan **Sistem Manajemen Skuad Manchester United berbasis C++** yang dilengkapi dengan fitur login dan registrasi akun sebelum pengguna dapat mengakses menu utama. Sistem dirancang untuk mengelola data pemain serta interaksi antara admin dan user dalam satu program berbasis console.

Saat program dijalankan, pengguna dapat memilih untuk login atau registrasi akun baru. Sistem memberikan maksimal 3 kali percobaan login. Jika login gagal sebanyak 3 kali, maka program akan berhenti secara otomatis. Jika login berhasil, pengguna akan diarahkan ke dashboard sesuai dengan peran masing-masing.

Program memiliki dua peran pengguna dengan hak akses berbeda, yaitu:

- **Admin**, dapat merekrut pemain baru, melihat daftar skuad lengkap beserta harga, memperbarui harga pasar pemain, memperbarui rating pemain, serta menjual atau menghapus pemain dari daftar skuad.
- **User**, hanya dapat melihat daftar skuad dan memberikan rating terhadap pemain yang tersedia.

Setiap fitur pada menu utama dipisahkan ke dalam subprogram (fungsi/prosedur) tersendiri, seperti prosesLogin, registrasiAkun, rekrutPemain, updateHarga, updateRatingAdmin, jualPemain, berikanRatingUser, dan tampilkanSkuad. Seluruh fungsi tersebut dipanggil dari fungsi main maupun fungsi lain menggunakan konsep **predefined process**, sehingga program menjadi lebih terstruktur dan modular.

Program ini juga menerapkan beberapa konsep pemrograman lanjutan, yaitu:

- **Pointer pada struct**, digunakan untuk mengakses dan memanipulasi data pemain secara langsung melalui alamat memori menggunakan operator panah (->), seperti pada fungsi getPemainByIndex.
- **Dereference operator (*)**, digunakan untuk mengubah nilai variabel secara langsung melalui alamat memori, contohnya pada fungsi terapkanHargaBaru dan hitungUlangRating.
- **Address-of operator (&)**, digunakan pada parameter fungsi untuk mengirim variabel secara reference sehingga perubahan di dalam fungsi akan mempengaruhi nilai asli, seperti pada fungsi rekrutPemain, jualPemain, dan registrasiAkun.

Program menggunakan berbagai struktur kontrol seperti:

- **Perulangan while**, untuk menjalankan menu utama dan proses login.
- **Perulangan do-while**, untuk validasi input agar tidak kosong.
- **Percabangan if-else**, untuk validasi login, pengecekan peran pengguna, serta pengambilan keputusan dalam menu.

Selain itu, program menggunakan **struct** untuk mengelompokkan data, yaitu PemainMU dan DataTeknis untuk data pemain, serta AkunPengguna untuk data akun. Data disimpan dalam bentuk **array statis** sebagai penyimpanan sementara selama program berjalan.

Program dilengkapi fitur sorting khusus admin melalui menu ke-6, dengan tiga metode pengurutan yang berbeda.

1. **Bubble Sort** mengurutkan nama pemain secara descending (Z ke A) dengan cara membandingkan dua elemen berdekatan dan menukarnya jika belum sesuai urutan. Terdapat optimasi berupa flag adaPertukaran yang menghentikan proses lebih awal jika tidak ada pertukaran dalam satu lintasan penuh.
2. **Selection Sort** mengurutkan harga pasar secara ascending (termurah ke termahal) dengan mencari elemen terkecil dari sisa data yang belum terurut, lalu menukarnya ke posisi paling depan secara berulang.
3. **Insertion Sort** mengurutkan rating pemain secara descending (tertinggi ke terendah) dengan mengambil satu elemen sebagai kunci dan menyisipkannya ke posisi yang tepat di antara elemen sebelumnya yang sudah terurut.

Program menampilkan tabel data sebelum dan sesudah sorting agar admin dapat membandingkan hasilnya secara langsung.

3. Source Code

```
#include <iostream>
#include <string>
#include <iomanip>
#include <cstdlib>

using namespace std;

struct DataTeknis {
    float     ratingPemain;
    long long hargaPasarUSD;
    int       jumlahPenilai;
    float     totalSkorRating;
```

```
};

struct PemainMU {
    string    namaLengkap;
    string    klubAsal;
    string    negaraAsal;
    DataTeknis performa;
};

struct AkunPengguna {
    string username;
    string password;
    string peran;
};

const int MAKSIMAL_DATA = 100;
const int MAKSIMAL_AKUN = 50;

PemainMU     daftarPemain[MAKSIMAL_DATA];
AkunPengguna databaseAkun[MAKSIMAL_AKUN];
int jumlahDataPemain = 0;
int jumlahAkun       = 0;


void cetakGaris(char karakter, int lebar) {
    cout << setfill(karakter) << setw(lebar) << karakter << setfill(' ') <<
endl;
}

PemainMU* getPemainByIndex(PemainMU daftarPemain[], int jumlahPemain, int
indeks) {
    if (indeks < 0 || indeks >= jumlahPemain) return nullptr;
    return &daftarPemain[indeks];
}

void terapkanHargaBaru(long long *hargaPasarTarget, long long hargaBaru) {
    cout << "\n  [Pointer] Alamat memori harga asli : " << hargaPasarTarget
<< endl;
    *hargaPasarTarget = hargaBaru;
    cout << "  [Pointer] Harga berhasil diubah via dereference ke: $" <<
*hargaPasarTarget << endl;
}

void hitungUlangRating(float *totalSkorTarget,
                       int   *jumlahPenilaiTarget,
                       float *ratingPemainTarget,
                       float  skorBaru) {
    *totalSkorTarget     += skorBaru;
    *jumlahPenilaiTarget += 1;
```

```
    *ratingPemainTarget   = *totalSkorTarget /
(float)(*jumlahPenilaiTarget);
}

void bubbleSortNamaDescending(PemainMU daftarPemain[], int jumlahPemain) {
    bool adaPertukaran;

    for (int lintasan = 0; lintasan < jumlahPemain - 1; lintasan++) {
        adaPertukaran = false;

        for (int posisi = 0; posisi < jumlahPemain - lintasan - 1; posisi++)
{
            if (daftarPemain[posisi].namaLengkap < daftarPemain[posisi +
1].namaLengkap) {
                PemainMU pemainSementara    = daftarPemain[posisi];
                daftarPemain[posisi]        = daftarPemain[posisi + 1];
                daftarPemain[posisi + 1]    = pemainSementara;
                adaPertukaran = true;
            }
        }

        if (adaPertukaran == false) break;
    }
}

void selectionSortHargaAscending(PemainMU daftarPemain[], int jumlahPemain)
{

    for (int posisiAwal = 0; posisiAwal < jumlahPemain - 1; posisiAwal++) {
        int indeksHargaTerkecil = posisiAwal;

        for (int pembanding = posisiAwal + 1; pembanding < jumlahPemain;
pembanding++) {
            if (daftarPemain[pembanding].performa.hargaPasarUSD <
                daftarPemain[indeksHargaTerkecil].performa.hargaPasarUSD) {
                indeksHargaTerkecil = pembanding;
            }
        }

        if (indeksHargaTerkecil != posisiAwal) {
            PemainMU pemainSementara           = daftarPemain[posisiAwal];
            daftarPemain[posisiAwal]           =
daftarPemain[indeksHargaTerkecil];
            daftarPemain[indeksHargaTerkecil]  = pemainSementara;
        }
    }
}

void insertionSortRatingDescending(PemainMU daftarPemain[], int
```

```
jumlahPemain) {

    for (int indeksKunci = 1; indeksKunci < jumlahPemain; indeksKunci++) {
        PemainMU pemainKunci = daftarPemain[indeksKunci];
        int posisiGeser      = indeksKunci - 1;

        while (posisiGeser >= 0 &&
               daftarPemain[posisiGeser].performa.ratingPemain <
pemainKunci.performa.ratingPemain) {
            daftarPemain[posisiGeser + 1] = daftarPemain[posisiGeser];
            posisiGeser--;
        }

        daftarPemain[posisiGeser + 1] = pemainKunci;
    }
}


void menuSortingAdmin(PemainMU daftarPemain[], int jumlahPemain) {
    system("cls");
    cout << "========================================" << endl;
    cout << "         MENU SORTING DATA PEMAIN       " << endl;
    cout << "========================================" << endl;

    if (jumlahPemain == 0) {
        cout << "Belum ada data pemain di skuad!" << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }

    cout << "1. Bubble Sort  - Nama Pemain (Descending / Z ke A)" << endl;
    cout << "2. Selection Sort - Harga Pasar (Ascending / Termurah ke
Termahal)" << endl;
    cout << "3. Insertion Sort - Rating Pemain (Descending / Tertinggi ke
Terendah)" << endl;
    cout << "4. Kembali" << endl;
    cout << "----------------------------------------" << endl;

    int pilihanSorting;
    cout << "Pilih metode sorting: "; cin >> pilihanSorting;

    if (pilihanSorting == 4) return;

    cout << "\n--- DATA SEBELUM SORTING ---" << endl;
    cetakGaris('=', 75);
    cout << left << setw(4) << "No"
         << setw(25) << "Nama Pemain"
         << setw(20) << "Klub Asal"
         << setw(10) << "Rating"
```

```
         << setw(16) << "Harga (USD)" << endl;
    cetakGaris('-', 75);
    for (int i = 0; i < jumlahPemain; i++) {
        PemainMU *pemainPtr = getPemainByIndex(daftarPemain, jumlahPemain,
i);
        cout << left << setw(4)  << i + 1
             << setw(25) << pemainPtr->namaLengkap
             << setw(20) << pemainPtr->klubAsal
             << setw(10) << fixed << setprecision(1) <<
pemainPtr->performa.ratingPemain
             << "$" << pemainPtr->performa.hargaPasarUSD << endl;
    }

    string namaMetode;
    string kriteriaSorting;

    if (pilihanSorting == 1) {
        bubbleSortNamaDescending(daftarPemain, jumlahPemain);
        namaMetode      = "BUBBLE SORT";
        kriteriaSorting = "Nama Pemain - Descending (Z ke A)";
    } else if (pilihanSorting == 2) {
        selectionSortHargaAscending(daftarPemain, jumlahPemain);
        namaMetode      = "SELECTION SORT";
        kriteriaSorting = "Harga Pasar - Ascending (Termurah ke Termahal)";
    } else if (pilihanSorting == 3) {
        insertionSortRatingDescending(daftarPemain, jumlahPemain);
        namaMetode      = "INSERTION SORT";
        kriteriaSorting = "Rating Pemain - Descending (Tertinggi ke
Terendah)";
    } else {
        cout << "\nPilihan tidak valid!" << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }
    cout << "\n--- DATA SESUDAH SORTING [" << namaMetode << "] ---" << endl;
    cout << "Kriteria : " << kriteriaSorting << endl;
    cetakGaris('=', 75);
    cout << left << setw(4) << "No"
         << setw(25) << "Nama Pemain"
         << setw(20) << "Klub Asal"
         << setw(10) << "Rating"
         << setw(16) << "Harga (USD)" << endl;
    cetakGaris('-', 75);
    for (int i = 0; i < jumlahPemain; i++) {
        PemainMU *pemainPtr = getPemainByIndex(daftarPemain, jumlahPemain,
i);
        cout << left << setw(4)  << i + 1
             << setw(25) << pemainPtr->namaLengkap
             << setw(20) << pemainPtr->klubAsal
```

```
            << setw(10) << fixed << setprecision(1) <<
pemainPtr->performa.ratingPemain
            << "$" << pemainPtr->performa.hargaPasarUSD << endl;
    }
    cetakGaris('=', 75);

    cout << "\n[OK] Data berhasil diurutkan menggunakan " << namaMetode <<
"!";
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}


void rekrutPemain(PemainMU daftarPemain[], int &jumlahPemain) {
    system("cls");
    cin.ignore();
    string    namaLengkap, klubAsal, negaraAsal;
    long long hargaPasarUSD;

    cout << "--- FORM REKRUTMEN ---" << endl;
    do {
        cout << "Nama Lengkap  : "; getline(cin, namaLengkap);
        if (namaLengkap.empty()) cout << "Nama tidak boleh kosong!\n";
    } while (namaLengkap.empty());
    do {
        cout << "Klub Asal     : "; getline(cin, klubAsal);
        if (klubAsal.empty()) cout << "Klub tidak boleh kosong!\n";
    } while (klubAsal.empty());
    do {
        cout << "Negara Asal   : "; cin >> negaraAsal;
        if (negaraAsal.empty()) cout << "Negara tidak boleh kosong!\n";
    } while (negaraAsal.empty());
    cout << "Harga (USD)   : $"; cin >> hargaPasarUSD;

    PemainMU *pemainBaru                      = &daftarPemain[jumlahPemain];
    pemainBaru->namaLengkap                   = namaLengkap;
    pemainBaru->klubAsal                      = klubAsal;
    pemainBaru->negaraAsal                    = negaraAsal;
    pemainBaru->performa.hargaPasarUSD        = hargaPasarUSD;
    pemainBaru->performa.ratingPemain         = 0.0f;
    pemainBaru->performa.jumlahPenilai        = 0;
    pemainBaru->performa.totalSkorRating      = 0.0f;

    jumlahPemain++;

    cout << "\n[&] Alamat jumlahPemain di rekrutPemain : " << &jumlahPemain
<< endl;
    cout << "Berhasil merekrut " << pemainBaru->namaLengkap << "! Tekan
Enter...";
    cin.ignore(); cin.get();
```

```
}

void jualPemain(PemainMU daftarPemain[], int &jumlahPemain) {
    system("cls");
    cout << "--- JUAL / HAPUS PEMAIN ---" << endl;
    if (jumlahPemain == 0) {
        cout << "Belum ada pemain di skuad." << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }

    cetakGaris('=', 55);
    for (int i = 0; i < jumlahPemain; i++) {
        PemainMU *pemainPtr = getPemainByIndex(daftarPemain, jumlahPemain,
i);
        cout << left << setw(4) << i + 1
             << setw(28) << pemainPtr->namaLengkap
             << "$" << pemainPtr->performa.hargaPasarUSD << endl;
    }
    cetakGaris('-', 55);

    int nomorPemain;
    cout << "Pilih nomor pemain yang dijual: "; cin >> nomorPemain;

    if (nomorPemain >= 1 && nomorPemain <= jumlahPemain) {
        string konfirmasi;
        PemainMU *pemainTerpilih = getPemainByIndex(daftarPemain,
jumlahPemain, nomorPemain - 1);
        cout << "Yakin ingin menjual " << pemainTerpilih->namaLengkap << "?
(ya/tidak): ";
        cin >> konfirmasi;

        if (konfirmasi == "ya" || konfirmasi == "Ya" || konfirmasi == "YA")
{
            string namaTerjual = pemainTerpilih->namaLengkap;
            for (int i = nomorPemain - 1; i < jumlahPemain - 1; i++) {
                daftarPemain[i] = daftarPemain[i + 1];
            }
            jumlahPemain--;
            cout << "\n[&] Alamat jumlahPemain di jualPemain : " <<
&jumlahPemain << endl;
            cout << namaTerjual << " berhasil dijual!" << endl;
        } else {
            cout << "\nPenjualan dibatalkan." << endl;
        }
    } else {
        cout << "\nNomor pemain tidak valid!" << endl;
    }
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
```

```
}

void updateHarga(PemainMU daftarPemain[], int jumlahPemain) {
    system("cls");
    cout << "--- UPDATE HARGA PEMAIN ---" << endl;
    if (jumlahPemain == 0) {
        cout << "Belum ada pemain di skuad." << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }

    cetakGaris('=', 55);
    for (int i = 0; i < jumlahPemain; i++) {
        PemainMU *pemainPtr = getPemainByIndex(daftarPemain, jumlahPemain,
i);
        cout << left << setw(4) << i + 1
             << setw(28) << pemainPtr->namaLengkap
             << "$" << pemainPtr->performa.hargaPasarUSD << endl;
    }
    cetakGaris('-', 55);

    int       nomorPemain;
    long long hargaBaru;
    cout << "Pilih nomor pemain : "; cin >> nomorPemain;

    if (nomorPemain >= 1 && nomorPemain <= jumlahPemain) {
        cout << "Harga baru (USD)   : $"; cin >> hargaBaru;
        PemainMU *pemainTerpilih = getPemainByIndex(daftarPemain,
jumlahPemain, nomorPemain - 1);
        terapkanHargaBaru(&pemainTerpilih->performa.hargaPasarUSD,
hargaBaru);
    } else {
        cout << "\nNomor pemain tidak valid!" << endl;
    }
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}

void updateRatingAdmin(PemainMU daftarPemain[], int jumlahPemain) {
    system("cls");
    cout << "--- UPDATE RATING PEMAIN (ADMIN) ---" << endl;
    if (jumlahPemain == 0) {
        cout << "Belum ada pemain di skuad." << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }

    cetakGaris('=', 55);
    for (int i = 0; i < jumlahPemain; i++) {
        PemainMU *pemainPtr = getPemainByIndex(daftarPemain, jumlahPemain,
```

```
i);
        cout << left << setw(4)  << i + 1
             << setw(28) << pemainPtr->namaLengkap
             << "Rating: " << fixed << setprecision(1)
             << pemainPtr->performa.ratingPemain << endl;
    }
    cetakGaris('-', 55);

    int   nomorPemain;
    float ratingBaru;
    cout << "Pilih nomor pemain  : "; cin >> nomorPemain;

    if (nomorPemain >= 1 && nomorPemain <= jumlahPemain) {
        cout << "Rating baru (0-10)  : "; cin >> ratingBaru;
        if (ratingBaru >= 0 && ratingBaru <= 10) {
            PemainMU *pemainTerpilih = getPemainByIndex(daftarPemain,
jumlahPemain, nomorPemain - 1);
            pemainTerpilih->performa.totalSkorRating = 0.0f;
            pemainTerpilih->performa.jumlahPenilai   = 0;
            hitungUlangRating(
                &pemainTerpilih->performa.totalSkorRating,
                &pemainTerpilih->performa.jumlahPenilai,
                &pemainTerpilih->performa.ratingPemain,
                ratingBaru
            );
            cout << "\n[*] Rating " << pemainTerpilih->namaLengkap
                 << " sekarang: " << fixed << setprecision(1)
                 << pemainTerpilih->performa.ratingPemain << endl;
        } else {
            cout << "\nRating harus antara 0 - 10!" << endl;
        }
    } else {
        cout << "\nNomor pemain tidak valid!" << endl;
    }
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}

void berikanRatingUser(PemainMU daftarPemain[], int jumlahPemain) {
    system("cls");
    cout << "--- BERIKAN RATING PEMAIN ---" << endl;
    if (jumlahPemain == 0) {
        cout << "Belum ada pemain di skuad." << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }

    cetakGaris('=', 65);
    for (int i = 0; i < jumlahPemain; i++) {
        PemainMU *pemainPtr = getPemainByIndex(daftarPemain, jumlahPemain,
```

```
i);
        cout << left << setw(4)  << i + 1
             << setw(28) << pemainPtr->namaLengkap
             << "Rating: " << fixed << setprecision(1)
             << pemainPtr->performa.ratingPemain
             << " (" << pemainPtr->performa.jumlahPenilai << " penilai)" <<
endl;
    }
    cetakGaris('-', 65);

    int   nomorPemain;
    float skorUser;
    cout << "Pilih nomor pemain : "; cin >> nomorPemain;

    if (nomorPemain >= 1 && nomorPemain <= jumlahPemain) {
        cout << "Beri rating (0-10) : "; cin >> skorUser;
        if (skorUser >= 0 && skorUser <= 10) {
            PemainMU *pemainTerpilih = getPemainByIndex(daftarPemain,
jumlahPemain, nomorPemain - 1);
            hitungUlangRating(
                &pemainTerpilih->performa.totalSkorRating,
                &pemainTerpilih->performa.jumlahPenilai,
                &pemainTerpilih->performa.ratingPemain,
                skorUser
            );
            cout << "\nTerima kasih! Rating " << pemainTerpilih->namaLengkap
                 << " sekarang: " << fixed << setprecision(1)
                 << pemainTerpilih->performa.ratingPemain << endl;
        } else {
            cout << "\nRating harus antara 0 - 10!" << endl;
        }
    } else {
        cout << "\nNomor pemain tidak valid!" << endl;
    }
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}


void cetakBarisPemainAdmin(PemainMU daftarPemain[], int indeks, int
totalPemain) {
    if (indeks >= totalPemain) return;
    PemainMU *pemainPtr = getPemainByIndex(daftarPemain, totalPemain,
indeks);
    cout << left << setw(4)  << indeks + 1
         << setw(22) << pemainPtr->namaLengkap
         << setw(18) << pemainPtr->klubAsal
         << setw(10) << fixed << setprecision(1) <<
pemainPtr->performa.ratingPemain
         << "$" << pemainPtr->performa.hargaPasarUSD << endl;
```

```
    cetakBarisPemainAdmin(daftarPemain, indeks + 1, totalPemain);
}

void cetakBarisPemainUser(PemainMU daftarPemain[], int indeks, int
totalPemain) {
    if (indeks >= totalPemain) return;
    PemainMU *pemainPtr = getPemainByIndex(daftarPemain, totalPemain,
indeks);
    cout << left << setw(4)  << indeks + 1
         << setw(25) << pemainPtr->namaLengkap
         << setw(12) << fixed << setprecision(1) <<
pemainPtr->performa.ratingPemain
         << setw(15) << pemainPtr->negaraAsal << endl;
    cetakBarisPemainUser(daftarPemain, indeks + 1, totalPemain);
}

void tampilkanSkuad(PemainMU daftarPemain[], int jumlahPemain) {
    system("cls");
    cout << "--- DAFTAR SKUAD (VIEW ONLY) ---" << endl;
    cetakGaris('=', 60);
    cout << left << setw(4) << "No" << setw(25) << "Nama"
         << setw(12) << "Rating" << setw(15) << "Negara" << endl;
    cetakGaris('-', 60);
    cetakBarisPemainUser(daftarPemain, 0, jumlahPemain);
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}

void tampilkanSkuad(PemainMU daftarPemain[], int jumlahPemain, bool
tampilHarga) {
    system("cls");
    cout << "--- DAFTAR SKUAD MANCHESTER UNITED ---" << endl;
    cetakGaris('=', 80);
    cout << left << setw(4) << "No" << setw(22) << "Nama"
         << setw(18) << "Asal Klub" << setw(10) << "Rating"
         << setw(15) << "Harga (USD)" << endl;
    cetakGaris('-', 80);
    cetakBarisPemainAdmin(daftarPemain, 0, jumlahPemain);
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}


bool cekDuplikatUsername(AkunPengguna databaseAkun[], int jumlahAkun, string
username) {
    for (int i = 0; i < jumlahAkun; i++) {
        if (databaseAkun[i].username == username) return true;
    }
    return false;
}
```

```
void registrasiAkun(AkunPengguna databaseAkun[], int &jumlahAkun) {
    system("cls");
    cout << "--- REGISTRASI USER BARU ---" << endl;
    if (jumlahAkun >= MAKSIMAL_AKUN) {
        cout << "Database akun penuh!" << endl;
        cout << "\nTekan Enter..."; cin.ignore(); cin.get();
        return;
    }

    string usernameBaru, passwordBaru;
    do {
        cout << "Username : "; cin >> usernameBaru;
        if (usernameBaru.empty()) cout << "Username tidak boleh kosong!\n";
    } while (usernameBaru.empty());

    if (cekDuplikatUsername(databaseAkun, jumlahAkun, usernameBaru)) {
        cout << "\nError: Username sudah terdaftar!" << endl;
    } else {
        do {
            cout << "Password : "; cin >> passwordBaru;
            if (passwordBaru.empty()) cout << "Password tidak boleh
kosong!\n";
        } while (passwordBaru.empty());

        databaseAkun[jumlahAkun].username = usernameBaru;
        databaseAkun[jumlahAkun].password = passwordBaru;
        databaseAkun[jumlahAkun].peran    = "user";
        jumlahAkun++;

        cout << "\n[&] Alamat jumlahAkun di registrasiAkun : " <<
&jumlahAkun << endl;
        cout << "Registrasi Berhasil! Silakan Login." << endl;
    }
    cout << "\nTekan Enter..."; cin.ignore(); cin.get();
}

AkunPengguna prosesLogin(AkunPengguna databaseAkun[], int jumlahAkun, bool
&statusLogin) {
    AkunPengguna sesiAktif;
    int          jumlahPercobaan = 0;

    while (jumlahPercobaan < 3) {
        system("cls");
        string inputUsername, inputPassword;
        cout << "--- FORM LOGIN (Percobaan: " << jumlahPercobaan + 1 << "/3)
---" << endl;
        cout << "Username : "; cin >> inputUsername;
        cout << "Password : "; cin >> inputPassword;
```

```
        for (int i = 0; i < jumlahAkun; i++) {
            if (inputUsername == databaseAkun[i].username &&
                inputPassword == databaseAkun[i].password) {
                sesiAktif   = databaseAkun[i];
                statusLogin = true;
                break;
            }
        }
        if (statusLogin) {
            cout << "\nLogin Berhasil! Tekan Enter...";
            cin.ignore(); cin.get();
            break;
        } else {
            jumlahPercobaan++;
            if (jumlahPercobaan < 3) {
                cout << "\nLogin Gagal! Username/Password salah.";
                cin.ignore(); cin.get();
            }
        }
    }
    return sesiAktif;
}


void tampilkanMenuAdmin() {
    cout << "1. Rekrut Pemain Baru"          << endl;
    cout << "2. Lihat Daftar Skuad"          << endl;
    cout << "3. Update Harga"                << endl;
    cout << "4. Update Rating"               << endl;
    cout << "5. Jual Pemain"                 << endl;
    cout << "6. Sorting Data Pemain   [NEW]" << endl;
    cout << "7. Logout"                      << endl;
}

void tampilkanMenuUser() {
    cout << "1. Lihat Daftar Skuad"    << endl;
    cout << "2. Berikan Rating Pemain" << endl;
    cout << "3. Logout"               << endl;
}

int main() {
    databaseAkun[jumlahAkun++] = {"nabil", "024", "admin"};
    databaseAkun[jumlahAkun++] = {"ewok",  "024", "user"};

    AkunPengguna sesiLogin;
    bool         programBerjalan = true;

    while (programBerjalan) {
        system("cls");
```

```
        int pilihanLayarAwal;
        cout << "========================================" << endl;
        cout << "    MANCHESTER UNITED MANAGER SYSTEM    " << endl;
        cout << "========================================" << endl;
        cout << "1. Login ke Sistem"                  << endl;
        cout << "2. Registrasi Akun Baru (User Only)" << endl;
        cout << "3. Tutup Program"                    << endl;
        cout << "Pilihan: "; cin >> pilihanLayarAwal;

        if (pilihanLayarAwal == 2) {
            registrasiAkun(databaseAkun, jumlahAkun);
            continue;
        }

        if (pilihanLayarAwal == 3) {
            system("cls");
            cout << "Program dihentikan. Glory Glory Man United!" << endl;
            programBerjalan = false;
            break;
        }

        if (pilihanLayarAwal == 1) {
            bool statusLogin = false;
            sesiLogin = prosesLogin(databaseAkun, jumlahAkun, statusLogin);

            if (!statusLogin) {
                system("cls");
                cout << "========================================" << endl;
                cout << "   AKSES DITOLAK! 3x Percobaan Gagal   " << endl;
                cout << "========================================" << endl;
                programBerjalan = false;
                break;
            }

            bool tetapDiMenu = true;
            while (tetapDiMenu) {
                system("cls");
                cout << "=== DASHBOARD "
                     << (sesiLogin.peran == "admin" ? "OFFICIAL ADMIN" :
"SUPPORTER ZONE")
                     << " ===" << endl;
                cout << "User Aktif : " << sesiLogin.username << endl;
                cout << "----------------------------------------" << endl;

                if (sesiLogin.peran == "admin") {
                    tampilkanMenuAdmin();
                } else {
                    tampilkanMenuUser();
                }
```

```
                int pilihanMenu;
                cout << "\nPilih Opsi: "; cin >> pilihanMenu;

                if (sesiLogin.peran == "admin") {
                    if      (pilihanMenu == 1) rekrutPemain(daftarPemain,
jumlahDataPemain);
                    else if (pilihanMenu == 2) tampilkanSkuad(daftarPemain,
jumlahDataPemain, true);
                    else if (pilihanMenu == 3) updateHarga(daftarPemain,
jumlahDataPemain);
                    else if (pilihanMenu == 4)
updateRatingAdmin(daftarPemain, jumlahDataPemain);
                    else if (pilihanMenu == 5) jualPemain(daftarPemain,
jumlahDataPemain);
                    else if (pilihanMenu == 6)
menuSortingAdmin(daftarPemain, jumlahDataPemain); // SORTING
                    else if (pilihanMenu == 7) tetapDiMenu = false;
                } else {
                    if      (pilihanMenu == 1) tampilkanSkuad(daftarPemain,
jumlahDataPemain);
                    else if (pilihanMenu == 2)
berikanRatingUser(daftarPemain, jumlahDataPemain);
                    else if (pilihanMenu == 3) tetapDiMenu = false;
                }
            }
        }
    }
    return 0;
}
```

4. Hasil Output

```
=========================================
    MANCHESTER UNITED MANAGER SYSTEM
=========================================
1. Login ke Sistem
2. Registrasi Akun Baru (User Only)
3. Tutup Program
Pilihan:
```

Gambar 4.1
<Tampilan Menu Awal setelah Program djalankanr>

```
--- FORM LOGIN (Percobaan: 1/3) ---
Username : nabil
Password : 024

Login Berhasil! Tekan Enter...
```

Gambar 4.2
<Tampilan menu 1 (login)>

```
=== DASHBOARD OFFICIAL ADMIN ===
User Aktif: nabil
---------------------------------------
1. Rekrut Pemain Baru
2. Lihat Daftar Skuad
3. Update Harga
4. Update Rating
5. Jual Pemain
6. Logout

Pilih Opsi:
```

Gambar 4.3
<Tampilan menu admin>

```
--- FORM REKRUTMEN ---
Nama Lengkap: Benjamin Sesko
Klub Asal   : RB Leipzig
Negara      : Slovenia
Harga (USD) : 3400000

Berhasil Rekrut! Tekan Enter...
```

Gambar 4.4
<Tampilan menu 1 admin rekrut pemain (create)>

```
--- DAFTAR SKUAD MANCHESTER UNITED ---
==============================================================================
No  Nama              Asal          Rating    Harga
------------------------------------------------------------------------------
1   Bryan Mbeumo      Brentford     0.0       $2950000
2   Matheus Cunha     Watford       0.0       $3100000
3   Benjamin Sesko    RB Leipzig    0.0       $3400000

Tekan Enter...
```

Gambar 4.5
<Tampilan menu 2 admin lihat daftar skuad (read)>

```
--- UPDATE HARGA PEMAIN ---
==================================================
1   Bryan Mbeumo            $2950000
2   Matheus Cunha           $3100000
3   Benjamin Sesko          $3400000
--------------------------------------------------
Pilih nomor pemain: 1
Harga baru (USD)  : 3300000
```

Gambar 4.6
<Tampilan menu 3 admin (update) harga pemain>

```
--- UPDATE RATING PEMAIN (ADMIN) ---
==================================================
1   Bryan Mbeumo            Rating: 0.0
2   Matheus Cunha           Rating: 0.0
3   Benjamin Sesko          Rating: 0.0
--------------------------------------------------
Pilih nomor pemain : 1
Rating baru (0-10) : 8.7
```

Gambar 4.7
<Tampilan Beri Rating Pemain (update)>

```
--- DAFTAR SKUAD MANCHESTER UNITED ---
==============================================================================
No  Nama                Asal          Rating    Harga
------------------------------------------------------------------------------
1   Bryan Mbeumo        Brentford     8.7       $3300000
2   Matheus Cunha       Watford       9.0       $3100000
3   Benjamin Sesko      RB Leipzig    9.3       $3400000

Tekan Enter...
```

Gambar 4.8
<Tampilan skuad setelah di update>

```
--- JUAL / HAPUS PEMAIN ---
==================================================
1   Bryan Mbeumo            $3300000
2   Matheus Cunha           $3100000
3   Benjamin Sesko          $3400000
--------------------------------------------------
Pilih nomor pemain yang dijual: 3
```

Gambar 4.9
<Tampilan pemain yang ingin dijual (delete)>

```
--- JUAL / HAPUS PEMAIN ---
=====================================================
1   Bryan Mbeumo              $3300000
2   Matheus Cunha             $3100000
3   Benjamin Sesko            $3400000
-----------------------------------------------------
Pilih nomor pemain yang dijual: 3
Yakin ingin menjual Benjamin Sesko? (ya/tidak): ya

Benjamin Sesko berhasil dijual!

Tekan Enter...
```

Gambar 4.9
<Tampilan pemain yang ingin dijual (delete)>

```
--- DAFTAR SKUAD MANCHESTER UNITED ---
=========================================================================
No  Nama                Asal           Rating    Harga
-------------------------------------------------------------------------
1   Bryan Mbeumo        Brentford      8.7       $3300000
2   Matheus Cunha       Watford        9.0       $3100000

Tekan Enter...
```

Gambar 4.10
<Tampilan skuad setelah dijual (delete)>

```
=========================================
         MENU SORTING DATA PEMAIN
=========================================
1. Bubble Sort  - Nama Pemain (Descending / Z ke A)
2. Selection Sort - Harga Pasar (Ascending / Termurah ke Termahal)
3. Insertion Sort - Rating Pemain (Descending / Tertinggi ke Terendah)
4. Kembali
-----------------------------------------
Pilih metode sorting:
```

Gambar 4.11
<Tampilan Menu 6 (Sorting)>

```
--- DATA SEBELUM SORTING ---
==============================================================================
No  Nama Pemain              Klub Asal           Rating    Harga (USD)
------------------------------------------------------------------------------
1   Bryan Mbeumo             Brentford           8.7       $2950000
2   Matheus Cunha            Watford             9.1       $31000000

--- DATA SESUDAH SORTING [BUBBLE SORT] ---
Kriteria : Nama Pemain - Descending (Z ke A)
==============================================================================
No  Nama Pemain              Klub Asal           Rating    Harga (USD)
------------------------------------------------------------------------------
1   Matheus Cunha            Watford             9.1       $31000000
2   Bryan Mbeumo             Brentford           8.7       $2950000
==============================================================================

[OK] Data berhasil diurutkan menggunakan BUBBLE SORT!
```

Gambar 4.12
<Tampilan Menu (Bubble Sort)>

```
--- DATA SEBELUM SORTING ---
==============================================================================
No  Nama Pemain              Klub Asal           Rating    Harga (USD)
------------------------------------------------------------------------------
1   Matheus Cunha            Watford             9.1       $31000000
2   Bryan Mbeumo             Brentford           8.7       $2950000

--- DATA SESUDAH SORTING [SELECTION SORT] ---
Kriteria : Harga Pasar - Ascending (Termurah ke Termahal)
==============================================================================
No  Nama Pemain              Klub Asal           Rating    Harga (USD)
------------------------------------------------------------------------------
1   Bryan Mbeumo             Brentford           8.7       $2950000
2   Matheus Cunha            Watford             9.1       $31000000
==============================================================================

[OK] Data berhasil diurutkan menggunakan SELECTION SORT!
```

Gambar 4.13
<Tampilan Menu (Selection Sort)>

```
--- DATA SEBELUM SORTING ---
==============================================================================
No  Nama Pemain              Klub Asal           Rating    Harga (USD)
------------------------------------------------------------------------------
1   Bryan Mbeumo             Brentford           8.7       $2950000
2   Matheus Cunha            Watford             9.1       $31000000

--- DATA SESUDAH SORTING [INSERTION SORT] ---
Kriteria : Rating Pemain - Descending (Tertinggi ke Terendah)
==============================================================================
No  Nama Pemain              Klub Asal           Rating    Harga (USD)
------------------------------------------------------------------------------
1   Matheus Cunha            Watford             9.1       $31000000
2   Bryan Mbeumo             Brentford           8.7       $2950000
==============================================================================

[OK] Data berhasil diurutkan menggunakan INSERTION SORT!
Tekan Enter...
```

Gambar 4.14
<Tampilan Menu (Insertion Sort)>

```
=== DASHBOARD SUPPORTER ZONE ===
User Aktif: ewok
----------------------------------------
1. Lihat Daftar Skuad
2. Berikan Rating Pemain
3. Logout

Pilih Opsi:
```

Gambar 4.16
<Tampilan menu user hanya bisa lihat dan kasih rating>

```
--- REGISTRASI USER BARU ---
Username : cuwiiii
Password : 420
```

Gambar 4.17
<Tampilan Registrasi akun>

```
--- REGISTRASI USER BARU ---
Username : ewok

Error: Username sudah terdaftar!

Tekan Enter untuk kembali...
```

Gambar 4.18
<Tampilan Registrasi akun jika username sama>

```
Program dihentikan. Glory Glory Man United!
PS D:\(KULIAH)\SEMESTER 2\Praktikum\p\praktikum-apl\posttest\posttest 2>
```

Gambar 4.19
<Tampilan Menu 3 (keluar dari program)>

5. Langkah-langkah GIT

5.1 GIT Init

```
Reinitialized existing Git repository in D:/(KULIAH)/SEMESTER 2/Praktikum/Praktikum SDA/post-t
est/POST-TEST SDA/POSTTEST SDA/POSTTEST 1/.git/
PS D:\(KULIAH)\SEMESTER 2\Praktikum\Praktikum SDA\post-test\POST-TEST SDA\POSTTEST SDA\POSTTES
T 1>
```

Gambar 5.1
<GIT Init>

Git inisialisasi kerepository

5.2 GIT Add

```
PS D:\(KULIAH)\SEMESTER 2\Praktikum\Praktikum APL\praktikum-apl\posttest\posttest 1> git add .
```

Gambar 5.2
<GIT Add.>

GIT Add digunakan untuk menambahkan semua file dan perubahan di folder saat ini ke staging area, agar siap untuk di-commit ke repository Git.

5.3 GIT Commit

```
Asus@Nabonkey MINGW64 /d/(KULIAH)/SEMESTER 2/Praktikum/p (main)
$ git commit -m "up"
[main 02534d2] up
 8 files changed, 658 insertions(+), 202 deletions(-)
 rename {praktikum-apl/.vscode => .vscode}/c_cpp_properties.json (100%)
 rename {praktikum-apl/.vscode => .vscode}/launch.json (100%)
 create mode 100644 .vscode/settings.json
 delete mode 100644 praktikum-apl/.vscode/settings.json
 create mode 100644 praktikum-apl/kelas/pertemuan 4/pertemuan4.cpp
 create mode 100644 praktikum-apl/posttest/posttest 4/2509106024-FERLYAHMADNABIL
-PT-4.cpp
 create mode 100644 praktikum-apl/posttest/posttest 4/2509106024-FERLYAHMADNABIL
-PT-4.exe
 create mode 100644 praktikum-apl/posttest/posttest 4/2509106024-FERLYAHMADNABIL
-PT-4.pdf
```

Gambar 5.3
<GIT Commit>

GIT Commit digunakan untuk menyimpan perubahan yang sudah ada di staging area ke dalam repository

5.4 GIT Push

```
Asus@Nabonkey MINGW64 /d/(KULIAH)/SEMESTER 2/Praktikum/p (main)
$ git push origin main
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 16 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (15/15), 1.06 MiB | 796.00 KiB/s, done.
Total 15 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/ferlynabil/Praktikum-APL.git
   e50a399..02534d2  main -> main
```

Gambar 5.4
<GIT Push>

GIT Push digunakan untuk mengirim (mengupload) commit dari repository lokal ke repository